

Análise Financeira de Filmes TMDb

Implementação de Data Lakehouse com Arquitetura Medallion

Este notebook implementa um pipeline de dados completo seguindo a arquitetura Medallion (Bronze, Silver, Gold) para análise financeira de filmes da base TMDb (The Movie Database). O projeto demonstra práticas de engenharia de dados incluindo:

- **Camada Bronze:** Ingestão de dados brutos do CSV
- **Camada Silver:** Limpeza, transformação e validação de dados
- **Camada Gold:** Modelagem dimensional (Star Schema) e análises de negócio
- **Quality Assurance:** Validação de qualidade e integridade dos dados
- **Business Intelligence:** Visualizações interativas e insights analíticos

Período de análise: 2000-2025

Dataset: TMDb Movies

```
In [0]: # - F (PySpark SQL functions): Funções para manipulação de dados no Spark
# - plotly.express e plotly.graph_objects: Criação de gráficos interativos
# - Window: Funções de janela para operações analíticas no Spark
# - pandas: Importado pela interação com as bibliotecas de visualização
```

```
from pyspark.sql import functions as F
import plotly.express as px
from pyspark.sql.window import Window
import plotly.graph_objects as go
import pandas as pd
```

```
In [0]: # Define o catálogo Unity Catalog a ser utilizado neste notebook.
# O catálogo 'mvp_puc' é onde todas as tabelas serão criadas e armazenadas.
```

```
spark.sql('USE CATALOG mvp_puc')
```

Out[0]: DataFrame[]

bronze

```
In [0]: # Parâmetros:
# - header=True: Primeira linha contém os nomes das colunas
# - sep=',': Separador de campos é vírgula
# - escape='\"': Caractere de escape para campos com vírgula dentro

movies = spark.read.csv('/Volumes/mvp_puc/default/tmdb/movies.csv', header=True, sep=',', escape='\"')
```

```
In [0]: movies.write.saveAsTable('bronze_movies', mode='overwrite')
```

```
In [0]: %sql
-- Mostra o histórico de transações da tabela bronze_movies.
-- Útil para auditoria e time travel (consultar versões anteriores dos dados).
```

```
DESCRIBE HISTORY bronze_movies
```

version	timestamp	userId	userName	operation	operationParameters
9	2026-09-21T12:41:04.000Z	75235409519145	gabi.abi9@gmail.com	CREATE OR REPLACE TABLE AS SELECT	Map(isV1SaveAsTableOverwrite -> true, partitionBy -> [], clusterBy -> [], description -> A tabela contém informações sobre filmes, incluindo detalhes como título, data de lançamento e orçamento. Possíveis usos incluem análise de tendências de bilheteira, avaliação de popularidade dos filmes e compreensão das preferências de gênero. Os dados também incluem avaliações e contagem de votos, que podem ser úteis para comparar o desempenho de diferentes filmes., isManaged -> true, properties -> {"delta.parquet.format.version":"2.12.0","delta.parquet.format.version.afe.internal":"2.12.0","delta.parquet.compression.codec":"zstd","delta.enableDeletionVectors":"true"}, statsOnLoad -> true)

8	2026-09-20T23:10:39.000Z	75235409519145	gabi.abi9@gmail.com	CREATE OR REPLACE TABLE AS SELECT	Map(isV1SaveAsTableOverwrite -> true, partitionBy -> [], clusterBy -> [], description -> A tabela contém informações sobre filmes, incluindo detalhes como título, data de lançamento e orçamento. Possíveis usos incluem análise de tendências de bilheteira, avaliação de popularidade dos filmes e compreensão das preferências de gênero. Os dados também incluem avaliações e contagem de votos, que podem ser úteis para comparar o desempenho de diferentes filmes., isManaged -> true, properties -> {"delta.parquet.format.version":"2.12.0","delta.parquet.format.version.afe.internal":"2.12.0","delta.parquet.compression.codec":"zstd","delta.enableDeletionVectors":"true"}, statsOnLoad -> true)
---	--------------------------	----------------	---------------------	-----------------------------------	---

data quality

```
In [0]: # Esta função gera um relatório detalhado de qualidade de dados, analisando:
# - Número de valores nulos por coluna
# - Número de strings vazias ou com apenas espaços
# - Número de valores distintos
# - Percentual de dados faltantes
#
# Parâmetros:
# - df: DataFrame do PySpark a ser analisado
# - dataset_name: Nome do dataset para identificação no relatório
# - key_column: Coluna chave (opcional, reservado para futuras validações)
```

```
def data_quality_report(df, dataset_name, key_column=None):
```

```
    # Conta o número total de linhas no dataset
    total_rows = df.count()
```

```
    # Lista para armazenar as métricas de qualidade de cada coluna
    report = []
```

```
    # Itera sobre cada coluna do DataFrame para coletar estatísticas
    for col_name in df.columns:
```

```
        # Conta valores nulos (NULL) na coluna
        null_count = df.filter(F.col(col_name).isNull()).count()
```

```
        # Conta strings vazias ou contendo apenas espaços em branco
        blank_count = df.filter(
            F.col(col_name).isNotNull() &
            (F.trim(F.col(col_name)) == "")
        ).count()
```

```
        # Conta o número de valores únicos na coluna
        distinct_count = df.select(col_name).distinct().count()
```

```
        # Calcula o total de dados faltantes e seu percentual
        missing_count = null_count + blank_count
        missing_pct = (
            (missing_count / total_rows) * 100
            if total_rows > 0 else 0
        )
```

```
        # Adiciona as métricas coletadas ao relatório
        report.append((
            dataset_name,
            col_name,
            str(df.schema[col_name].dataType),
```

```
        total_rows,
        null_count,
        blank_count,
        missing_count,
        round(missing_pct, 2),
        distinct_count
    ))

# Retorna um DataFrame PySpark com o relatório de qualidade
return spark.createDataFrame(
    report,
    [
        "dataset",
        "column",
        "data_type",
        "total_rows",
        "null_count",
        "blank_count",
        "missing_count",
        "missing_pct",
        "distinct_count"
    ]
)
```

```
In [0]: # Aplica a função de qualidade de dados ao DataFrame de filmes.
quality_movies = data_quality_report(
    movies,
    "movies",
    key_column="id"
)
display(quality_movies)
```

dataset	column	data_type	total_rows	null_count	blank_count	missing_count	missing_pct	distinct_count
movies	id	StringType()	9778	0	0	0	0.0	9778
movies	title	StringType()	9778	0	0	0	0.0	9163
movies	original_title	StringType()	9778	0	0	0	0.0	9236
movies	overview	StringType()	9778	160	0	160	1.64	9607
movies	release_date	StringType()	9778	68	0	68	0.7	6513
movies	runtime	StringType()	9778	8	0	8	0.08	235
movies	budget	StringType()	9778	8	0	8	0.08	652
movies	revenue	StringType()	9778	8	0	8	0.08	3857
movies	vote_average	StringType()	9778	8	0	8	0.08	2659
movies	vote count	StrinaType()	9778	9	0	9	0.09	3114

silver

```
In [0]: # Este processo inclui:
# - Seleção de colunas relevantes
# - Validação e conversão de tipos de dados
# - Filtragem de registros inválidos ou inconsistentes
# - Padronização de formatos (datas, números)

# Seleciona apenas as colunas necessárias para a análise
movies_selected = movies.select("id", "release_date", "budget", "revenue", "genres")

# 1. Conversão Inicial e Tratamento de Dados Numéricos
# Remove IDs inválidos (que não podem ser convertidos para inteiro)
df_elegivel = movies_selected.filter(F.col('id').try_cast('int').isNull()) \
    .withColumn('budget', F.col('budget').cast('double')) \
    .withColumn('revenue', F.col('revenue').cast('double'))

# 2. Tratamento de Datas
# Converte a coluna release_date para o tipo DATE
df_elegivel = df_elegivel.withColumn('release_date', F.to_date(F.col('release_date'))))

# 3. Aplicação de Filtros de Elegibilidade
# Mantém apenas filmes com dados financeiros válidos e data de lançamento
df_filtered = df_elegivel.filter(
    (F.col('revenue') > 0) & # Faturamento positivo
    (F.col('budget') > 0) & # Orçamento positivo
    (F.col('release_date').isNull()) # Data de lançamento presente
)

# 4. Criação de Colunas Temporais
# Extrai ano e mês da data de lançamento para facilitar análises temporais
df_filtered = df_filtered \
    .withColumn('year', F.year(F.col('release_date')))) \
    .withColumn('month', F.month(F.col('release_date'))))

# Filtra apenas filmes lançados entre 2000 e 2025
# Este período foi escolhido para focar em produções da mesma geração
df_filtered = df_filtered.filter(
    (F.col('year') >= 2000) & (F.col('year') <= 2025)
)

# 5. Ordenação por data de lançamento
# Organiza os dados cronologicamente para facilitar análises temporais
df_filtered = df_filtered.orderBy('release_date')
df_filtered.show(5)
```

```
+-----+-----+-----+-----+-----+-----+
| id|release_date| budget| revenue| genres|year|month|
+-----+-----+-----+-----+-----+-----+
| 1359| 2000-01-21|7000000.0| 3.4269748E7|Drama,Thriller,Crime|2000| 1|
|47816| 2000-03-01|6000000.0| 9000000.0|Action,Comedy,Rom...|2000| 3|
|25212| 2000-03-10| 800000.0| 145071.0|Drama,Action,Thri...|2000| 3|
| 9532| 2000-03-17| 2.3E7|1.12880294E8| Horror|2000| 3|
| 462| 2000-03-17| 5.2E7|2.56271286E8| Drama|2000| 3|
+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
```

```
In [0]: df_filtered.write.saveAsTable('silver_movies', mode='overwrite')
```

gold

```
In [0]: # Transforma a coluna de gêneros (formato CSV) em registros individuais.
# Cada filme pode ter múltiplos gêneros, resultando em várias linhas por filme.

# Divide a string de gêneros separada por vírgula em um array
split_genre = df_filtered.select(F.col('id').alias('movie_id'),'genres').withColumn('genres', F.split(F.col('genres'), ','))

# Explode o array, criando uma linha para cada gênero de cada filme
dim_genres = split_genre.select('movie_id','genres').withColumn('genres', F.explode(F.col('genres')))
dim_genres.show(5)
```

```
+-----+-----+
|movie_id|  genres|
+-----+-----+
|    1359|   Drama|
|    1359| Thriller|
|    1359|   Crime|
|   47816|  Action|
|   47816|  Comedy|
+-----+-----+
only showing top 5 rows
```

```
In [0]: # Lista todos os gêneros únicos presentes no dataset.
display(dim_genres.select('genres').distinct())
```

genres
Drama
Thriller
Crime
Action
Comedy
Romance
Mystery
Music
Horror
Adventure

```
In [0]: # Tabela fato contendo métricas financeiras dos filmes.
# Inclui cálculos de:
# - Profit (Lucro): Diferença entre receita e orçamento
# - ROI (Return on Investment): Retorno sobre investimento (receita / orçamento)

fact_movie_finance = df_filtered.select(F.col('id').alias('movie_id'),'release_date','budget', 'revenue') \
    .withColumn('profit', F.col('revenue') - F.col('budget')) \
    .withColumn('roi', F.col('revenue') / F.col('budget'))
fact_movie_finance.show(5)
```

```
+-----+-----+-----+-----+-----+-----+
|movie_id|release_date|  budget|   revenue|   profit|         roi|
+-----+-----+-----+-----+-----+-----+
|    1359|  2000-01-21|7000000.0| 3.4269748E7| 2.7269748E7|4.895678285714285|
|   47816|  2000-03-01|6000000.0| 9000000.0| 3000000.0|          1.5|
|   25212|  2000-03-10| 800000.0| 145071.0| -654929.0|    0.18133875|
|    9532|  2000-03-17|   2.3E7|1.12880294E8| 8.9880294E7|4.907838869565217|
|     462|  2000-03-17|   5.2E7|2.56271286E8|2.04271286E8|4.928293961538461|
+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
```

```
In [0]: # Tabela dimensional contendo informações sobre datas de lançamento.
# Facilita análises temporais ao fornecer atributos de data pré-calculados.
```

```
# Cria um mapeamento de número do mês para nome abreviado em português
month_mapping = F.create_map(
    F.lit(1), F.lit('Jan'), F.lit(2), F.lit('Fev'), F.lit(3), F.lit('Mar'),
    F.lit(4), F.lit('Abr'), F.lit(5), F.lit('Mai'), F.lit(6), F.lit('Jun'),
    F.lit(7), F.lit('Jul'), F.lit(8), F.lit('Ago'), F.lit(9), F.lit('Set'),
    F.lit(10), F.lit('Out'), F.lit(11), F.lit('Nov'), F.lit(12), F.lit('Dez')
)

# Cria a dimensão com datas únicas e seus atributos
dim_date = df_filtered.select('release_date', 'year', 'month').distinct() \
    .withColumn('month_name', month_mapping[F.col('month')])
dim_date.show(5)
```

```
+-----+-----+-----+-----+
|release_date|year|month|month_name|
+-----+-----+-----+-----+
|  2003-05-30|2003|   5|    Mai|
|  2000-09-01|2000|   9|     Set|
|  2005-10-26|2005|  10|     Out|
|  2007-05-19|2007|   5|    Mai|
|  2002-05-24|2002|   5|    Mai|
+-----+-----+-----+-----+
only showing top 5 rows
```

```
In [0]: # A camada Gold contém dados modelados em Star Schema, otimizados para análises de negócio e dashboards.
dim_genres.write.saveAsTable('gold_dim_genres', mode='overwrite')
fact_movie_finance.write.saveAsTable('gold_fact_movie_finance', mode='overwrite')
dim_date.write.saveAsTable('gold_dim_date', mode='overwrite')
```

```
In [0]: %sql
-- Define chaves primárias e relacionamentos (FK) entre as tabelas Gold no Unity Catalog
-- As constraints no Unity Catalog são informativas (não enforcement)

-- Alterar todas as colunas de chave primária para NOT NULL
ALTER TABLE mvp_puc.default.gold_fact_movie_finance
    ALTER COLUMN movie_id SET NOT NULL;

ALTER TABLE mvp_puc.default.gold_dim_date
    ALTER COLUMN release_date SET NOT NULL;

ALTER TABLE mvp_puc.default.gold_dim_genres
    ALTER COLUMN movie_id SET NOT NULL;

ALTER TABLE mvp_puc.default.gold_dim_genres
    ALTER COLUMN genres SET NOT NULL;

-- 1. Chave primária na tabela fato (um registro por filme)
ALTER TABLE mvp_puc.default.gold_fact_movie_finance
    ADD CONSTRAINT pk_gold_fact_movie_finance PRIMARY KEY (movie_id);

-- 2. Chave primária na dimensão de data (uma data distinta por linha)
ALTER TABLE mvp_puc.default.gold_dim_date
    ADD CONSTRAINT pk_gold_dim_date PRIMARY KEY (release_date);

-- 3. Chave primária composta na dimensão de gêneros (movie_id + genres)
ALTER TABLE mvp_puc.default.gold_dim_genres
    ADD CONSTRAINT pk_gold_dim_genres PRIMARY KEY (movie_id, genres);

-- 4. FK: dim_genres.movie_id -> fact_movie_finance.movie_id
ALTER TABLE mvp_puc.default.gold_dim_genres
    ADD CONSTRAINT fk_dim_genres_fact_movie_finance
    FOREIGN KEY (movie_id) REFERENCES mvp_puc.default.gold_fact_movie_finance(movie_id);

-- 5. FK: fact_movie_finance.release_date -> dim_date.release_date
ALTER TABLE mvp_puc.default.gold_fact_movie_finance
    ADD CONSTRAINT fk_fact_movie_finance_dim_date
    FOREIGN KEY (release_date) REFERENCES mvp_puc.default.gold_dim_date(release_date);
```

validação

```
In [0]: # Verifica se existem IDs de filmes duplicados na tabela fato.
# Uma tabela fato deve ter chaves únicas; duplicatas indicariam erro na ETL.
# Resultado esperado: nenhuma linha (tabela vazia)
```

```
validate_fact = fact_movie_finance \
    .groupBy("movie_id") \
    .count() \
    .filter(F.col("count") > 1)

validate_fact.show()

+-----+-----+
|movie_id|count|
+-----+-----+
+-----+-----+
```

```
In [0]: # Testa a integridade referencial entre a tabela fato e a dimensão de gêneros.
# Verifica se o relacionamento entre as tabelas funciona corretamente.
movie_genres_join = (
    fact_movie_finance.alias("f")
    .join(
        dim_genres.alias("g"),
        F.col("f.movie_id") == F.col("g.movie_id"),
        "inner"
    )
)

movie_genres_join.show(5)

+-----+-----+-----+-----+-----+-----+-----+-----+
|movie_id|release_date|budget|    revenue|    profit|          roi|movie_id|genres|
+-----+-----+-----+-----+-----+-----+-----+-----+
|      12|  2003-05-30|  9.4E7|9.40335536E8|8.46335536E8|10.003569531914893|      12|Family|
|      22|  2003-07-09|  1.4E8|6.55011224E8|5.15011224E8|         4.6786516|      22|Action|
|      24|  2003-10-10|  3.0E7|1.80906076E8|1.50906076E8|  6.030202533333333|      24|  Crime|
|      25|  2005-11-04|  7.2E7| 9.7076152E7| 2.5076152E7|  1.348279888888889|      25|   War|
|      35|  2007-07-25|  7.5E7|5.27068851E8|4.52068851E8|         7.02758468|      35|Family|
+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
```

```
In [0]: # Identifica filmes na tabela fato que não possuem gênero associado.
# Usa LEFT ANTI JOIN para encontrar registros na tabela fato sem correspondência na dimensão de gêneros.
# Útil para identificar possíveis problemas de qualidade de dados.
movies_without_genre = (
    fact_movie_finance.alias("f")
    .join(
        dim_genres.alias("g"),
        F.col("f.movie_id") == F.col("g.movie_id"),
        "left_anti"
    )
)

movies_without_genre.show()

+-----+-----+-----+-----+-----+-----+
|movie_id|release_date|  budget| revenue|  profit|    roi|
+-----+-----+-----+-----+-----+-----+
|  207490|  2000-05-05|1000000.0|428535.0|-571465.0|0.428535|
+-----+-----+-----+-----+-----+-----+
```

análise

```
In [0]: # Combina a tabela fato com as dimensões de gêneros e data.
# Esta view consolida todas as informações necessárias para análises financeiras por gênero ao longo do tempo.
# Serve como base para todas as análises e visualizações subsequentes.
genre_analysis = (
    fact_movie_finance.alias("f")
    .join(
        dim_genres.alias("g"),
        'movie_id',
        "inner"
    )
    .join(
        dim_date.alias("d"),
        F.col("f.release_date") == F.col("d.release_date"),
        "left"
    )
    .select(
        F.col("f.movie_id"),
        F.col("g.genres"),
        F.col("f.budget"),
        F.col("f.revenue"),
        F.col("f.profit"),
        F.col("f.roi"),
        F.col("d.year")
    )
)
```

```
In [0]: # Salva a view analítica como tabela para otimizar consultas futuras.
genre_analysis.write.saveAsTable('gold_genre_analysis', mode='overwrite')
```

1. Quais gêneros possuem maior volume de filmes?

```
In [0]: # Utiliza countDistinct para evitar contar o mesmo filme múltiplas vezes (um filme pode ter múltiplos gêneros).

# Agrega os dados contando filmes distintos por gênero
genre_vol = (
    genre_analysis
    .groupBy("genres")
    .agg(
        F.countDistinct("movie_id").alias("qtd_filmes")
    )
    .orderBy(F.desc("qtd_filmes"))
)

# Converte para Pandas para visualização com Plotly
genre_vol_pd = genre_vol.toPandas()

# Cria gráfico de barras interativo
genre_vol_fig = px.bar(
    genre_vol_pd,
    x="genres",
    y="qtd_filmes",
    barmode="group",
    title="Quantidade de Filmes por Gênero",
    labels={
        "qtd_filmes": "Qtd. Filmes",
        "genres": "Gênero"
    },
    text="qtd_filmes"
)

genre_vol_fig.show()
```

2. Quais gêneros apresentam maior faturamento médio?

```
In [0]: # Agrega calculando a média de receita e orçamento por gênero
genre_revenue = (
```

```
genre_analysis
.groupby("genres")
.agg(
    F.avg("revenue").alias("faturamento_medio"),
    F.avg("budget").alias("orcamento_medio")
)
.orderBy(F.desc("faturamento_medio"))
)

# Converte para Pandas para visualização
genre_revenue_pd = genre_revenue.toPandas()

# Cria gráfico de barras com formatação monetária
genre_revenue_fig = px.bar(
    genre_revenue_pd,
    x="genres",
    y=["faturamento_medio", "orcamento_medio"],
    barmode="group",
    title="Faturamento Médio e Orçamento Médio por Gênero",
    labels={
        "faturamento_medio": "Faturamento Médio",
        "orcamento_medio": "Orçamento Médio",
        "genres": "Gênero"
    },
    text_auto='$,.0f'
)

genre_revenue_fig.update_traces(textfont_color="black")
genre_revenue_fig.show()
```

3. Quais gêneros apresentam maior lucro médio?

```
In [0]: # Agrega calculando a média de Lucro por gênero
genre_profit = (
    genre_analysis
    .groupBy("genres")
    .agg(
        F.avg("profit").alias("lucro_medio")
    )
    .orderBy(F.desc("lucro_medio"))
)

# Converte para Pandas para visualização
genre_profit_pd = genre_profit.toPandas()

# Cria gráfico de barras com formatação monetária
genre_profit_fig = px.bar(
    genre_profit_pd,
    x="genres",
    y="lucro_medio",
    barmode="group",
    title="Lucro Médio por Gênero",
    labels={
        "lucro_medio": "Lucro Médio",
        "genres": "Gênero"
    },
    text_auto=True,
    text="lucro_medio"
)

# Formata os valores como moeda (dólar)
genre_profit_fig.update_traces(texttemplate='${text:,.0f}')
genre_profit_fig.show()
```

4. Quais gêneros apresentam maior ROI médio?

```
In [0]: # ROI = Receita / Orçamento
# Um ROI de 2.0 significa que o filme gerou o dobro do que foi investido.
# Esta métrica é especialmente útil para comparar a eficiência do investimento independentemente do tamanho do orçamento.

# Agrega calculando a média de ROI por gênero
# Filtra apenas orçamentos positivos para evitar divisão por zero
genre_roi = (
    genre_analysis
    .groupBy("genres")
    .agg(
        F.avg(
            F.when(F.col("budget") > 0, F.col("roi"))
        ).alias("roi_medio")
    )
    .orderBy(F.desc("roi_medio"))
)

# Converte para Pandas para visualização
genre_roi_pd = genre_roi.toPandas()

# Cria gráfico de barras com formatação de múltiplo
genre_roi_fig = px.bar(
    genre_roi_pd,
    x="genres",
    y="roi_medio",
    barmode="group",
    title="ROI Médio por Gênero",
    labels={
        "roi_medio": "ROI Médio (múltiplo)",
        "genres": "Gênero"
    },
    text_auto=True,
    text="roi_medio"
)

# Formata os valores como múltiplo (ex: 2.5x)
genre_roi_fig.update_traces(texttemplate='%{text:.1f}x')
genre_roi_fig.show()
```

5. Como a participação dos gêneros na produção cinematográfica evoluiu ao longo do tempo?

```
In [0]: # Agrega contando filmes distintos por gênero e ano
genre_by_year = genre_analysis.groupBy("genres", "year").agg(F.countDistinct("movie_id").alias("qtd_filmes")).orderBy(F.desc("qtd_filmes"))
genre_by_year.show(3)

+-----+-----+
| genres|year|qtd_filmes|
+-----+-----+
| Drama|2025|      56|
|Thriller|2025|      55|
| Drama|2024|      53|
+-----+-----+
only showing top 3 rows
```

```
In [0]: # Cria uma visualização dos três gêneros mais produzidos em cada ano para facilitar visualização.

# Define a especificação da janela para rankear gêneros por ano
window_spec = (
```

```
Window
.partitionBy("year") # Particiona por ano
.orderBy(F.desc("qtd_filmes")) # Ordena por quantidade decrescente
)

# Aplica o ranking e filtra apenas os top 3 de cada ano
top3_genres_by_year = (
    genre_by_year
    .withColumn("rank", F.row_number().over(window_spec))
    .filter(F.col("rank") <= 3) # Mantém apenas os 3 primeiros
    .drop("rank")
)

# Converte para Pandas para visualização
top3_genres_by_year_pd = top3_genres_by_year.toPandas()

# Cria gráfico de barras agrupadas por ano
fig = px.bar(
    top3_genres_by_year_pd,
    x="year",
    y="qtd_filmes",
    color="genres",
    barmode="group",
    title="Top 3 Gêneros dos Filmes por Ano",
    labels={
        "qtd_filmes": "Qtd. Filmes",
        "year": "Ano",
        "genres": "Gênero"
    },
    text="qtd_filmes"
)

# Configura o layout do gráfico
fig.update_layout(
    xaxis_title="Ano",
    yaxis_title="Qtd. Filmes",
    hovermode="x unified" # Unifica informações ao passar o mouse
)

fig.show()
```

In [0]: # A coluna "genres" contém os gêneros separados por vírgula (ex: "Action, Comedy").
Para filmes sem gênero (nulo ou vazio), a contagem é definida como 0.
O resultado é ordenado em ordem decrescente para identificar os filmes com maior diversidade de gêneros.

```
movies_genre_count = (
    df_filtered
    .withColumn(
        "qtd_generos",
        # Se genres for nulo ou uma string vazia/contendo apenas espaços,
        # define a quantidade de gêneros como 0.
        F.when(
            F.col("genres").isNull() | (F.trim(F.col("genres")) == ""),
            0
        ).otherwise(
            # Caso contrário, divide a string de gêneros por vírgula
            # e conta quantos elementos resultaram da divisão.
            F.size(
                F.split(F.col("genres"), ",")
            )
        )
    )
    .select(
        # Renomeia "id" para "movie_id" para manter o padrão do modelo
        F.col("id").alias("movie_id"),
        "release_date",
        "genres",
        "qtd_generos"
    )
    # Ordena pelos filmes com maior quantidade de gêneros primeiro
    .orderBy(F.desc("qtd_generos"))
)

movies_genre_count.show(5)
```

```
+-----+-----+-----+-----+
|movie_id|release_date|          genres|qtd_generos|
+-----+-----+-----+-----+
|  354279|  2018-03-26|Adventure,Fantasy...|          7|
|  82702|  2014-06-05|Adventure,Fantasy...|          6|
| 12610|  2001-08-10|Adventure,Animati...|          6|
|   1966|  2004-11-21|Adventure,Drama,A...|          6|
| 10947|  2006-01-20|Drama,Comedy,Musi...|          6|
+-----+-----+-----+-----+
only showing top 5 rows
```

Análises sobre o dataset

In [0]: # Esta análise utiliza um gráfico Candlestick (velas) para mostrar a distribuição estatística da quantidade de gêneros por filme ao longo dos anos.

```
# Primeiro, cria a estrutura de contagem de gêneros por filme
movies_genre_count = (
    df_filtered
    .withColumn(
        "qtd_generos",
        F.when(
            F.col("genres").isNull() | (F.trim(F.col("genres")) == ""),
            0
        ).otherwise(
            F.size(F.split(F.col("genres"), ","))
        )
    )
    .select(
        F.col("id").alias("movie_id"),
        "release_date",
        "genres",
        "qtd_generos"
    )
)

# Adiciona informação de ano juntando com a dimensão de data
movies_with_year = movies_genre_count.alias("f").join(dim_date.alias("d"),
    F.col("f.release_date") == F.col("d.release_date"),
    "left")

# Calcula estatísticas para o gráfico candlestick por ano
# Low/High: Mínimo e máximo de gêneros por filme
# Open/Close: Percentis 25 e 75 (quartis da distribuição)
candlestick_data = (
    movies_with_year
    .groupBy('year')
    .agg(
        F.min('qtd_generos').alias('low'),      # Mínimo
        F.max('qtd_generos').alias('high'),      # Máximo
        F.expr('percentile_approx(qtd_generos, 0.25)').alias('open'),  # 1º quartil
        F.expr('percentile_approx(qtd_generos, 0.75)').alias('close')  # 3º quartil
    )
    .orderBy('year')
)
```

```
# Converte para Pandas para visualização
candlestick_pd = candlestick_data.toPandas()

# Cria gráfico candlestick (velas)
# Este tipo de gráfico é ideal para mostrar distribuição estatística ao longo do tempo
fig = go.Figure(
    data=[
        go.Candlestick(
            x=candlestick_pd['year'],
            open=candlestick_pd['open'],
            high=candlestick_pd['high'],
            low=candlestick_pd['low'],
            close=candlestick_pd['close']
        )
    ]
)

# Configura o Layout do gráfico
fig.update_layout(
    title='Quantidade de Gêneros nos Filmes por Ano',
    xaxis_title='Ano',
    yaxis_title='Qtd. Gêneros',
    xaxis_rangeslider_visible=False # Remove o seletor de intervalo
)

fig.show()
```

In [0]:

```
# Identifica os pares de gêneros que mais frequentemente aparecem juntos.
# Esta análise utiliza um self-join para encontrar todas as combinações de gêneros dentro do mesmo filme.
# Útil para entender tendências de hibridização de gêneros no cinema.
```

```
genre_pairs = (
    genre_analysis.alias("g1")
    .join(
        genre_analysis.alias("g2"),
        # Junta filmes pelo ID e garante que não há duplicação de pares
        (F.col("g1.movie_id") == F.col("g2.movie_id")) &
        (F.col("g1.genres") < F.col("g2.genres")), # Evita duplicação: (A,B) = (B,A)
        "inner"
    )
    .groupBy(
        F.col("g1.genres").alias("genero_1"),
        F.col("g2.genres").alias("genero_2")
    )
    .agg(
        F.countDistinct("g1.movie_id").alias("qtd_filmes")
    )
    .orderBy(F.desc("qtd_filmes")) # Ordena pelos pares mais frequentes
)

display(genre_pairs)
```

genero_1	genero_2	qtd_filmes
Action	Adventure	349
Action	Thriller	325
Action	Science Fiction	247
Comedy	Family	244
Drama	Thriller	234
Crime	Thriller	219
Adventure	Family	219
Drama	Romance	218
Adventure	Comedy	216
Animation	Familv	215